

Sim-to-Real Learning of All Common Bipedal Gaits via Periodic Reward Composition

Jonah Siekmann*, Yesh Godse*, Alan Fern, Jonathan Hurst
Collaborative Robotics and Intelligent Systems Institute
Oregon State University
{siekmannj, godsey, afern, jhurst}@oregonstate.edu

Abstract—We study the problem of realizing the full spectrum of bipedal locomotion on a real robot with sim-to-real reinforcement learning (RL). A key challenge of learning legged locomotion is describing different gaits, via reward functions, in a way that is intuitive for the designer and specific enough to reliably learn the gait across different initial random seeds or hyperparameters. A common approach is to use reference motions (e.g. trajectories of joint positions) to guide learning. However, finding high-quality reference motions can be difficult and the trajectories themselves narrowly constrain the space of learned motion. At the other extreme, reference-free reward functions are often underspecified (e.g. move forward) leading to massive variance in policy behavior, or are the product of significant reward-shaping via trial-and-error, making them exclusive to specific gaits. In this work, we propose a reward-specification framework based on composing simple probabilistic periodic costs on basic forces and velocities. We instantiate this framework to define a parametric reward function with intuitive settings for all common bipedal gaits - standing, walking, hopping, running, and skipping. Using this function we demonstrate successful sim-to-real transfer of the learned gaits to the bipedal robot Cassie, as well as a generic policy that can transition between all of the two-beat gaits.

I. INTRODUCTION

Using reinforcement learning (RL) to learn all of the common bipedal gaits found in nature for a real robot is an unsolved problem. A key challenge of learning a specific locomotion gait via RL is to communicate the gait behavior through the reward function. In general, a specific gait can be viewed as a dynamic process that has a characteristic periodic structure, but is also able to flexibly adapt to moderate environment disturbances. This suggests two considerations when designing a gait reward function. First, the reward must be specific enough to produce the desired gait characteristic when optimized. Second, to account for the fact that there is uncertainty about the exact details of a gait in the context of specific terrain and dynamic conditions, the reward should not be overly constraining.

The common use of reference trajectories to specify gait-specific rewards, e.g., [1]–[5], partly addresses the first consideration above, but mostly ignores the second. In particular, a reference trajectory only captures a small part of the variation needed to realize a gait characteristic under varying conditions. Thus, attempting to adhere to such a trajectory can prevent learning a characteristic gait that is more robust and/or efficient, not to mention that deriving

* denotes equal contribution, order determined by coin toss

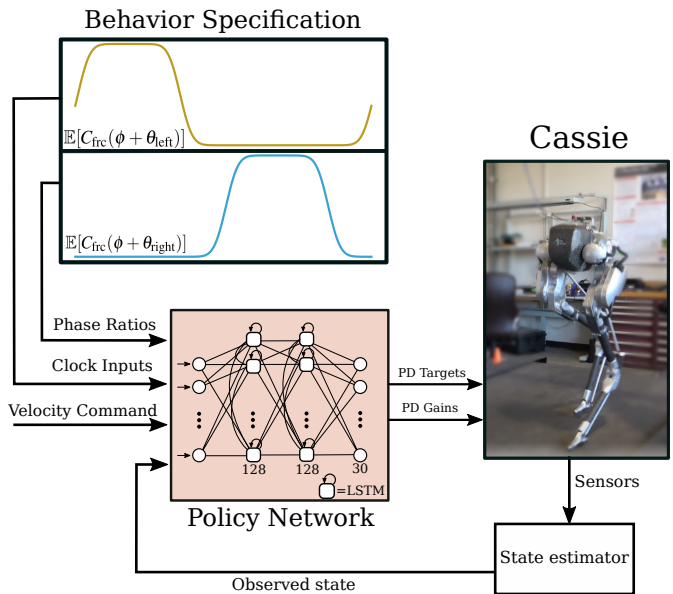


Fig. 1. In this work, we present a reward design framework which makes it easy to learn policies which can stand, walk, run, gallop, hop, and skip on hardware. We condition the reward function based on a number of gait parameters, and also provide these parameters to the LSTM policy, which outputs PD joint position targets and PD gains to the robot.

feasible reference trajectories for a particular desired gait can be very challenging in the first place.

Reference-free approaches to specifying reward functions for locomotion are often highly underspecified, for example, those used in the OpenAI Gym [6] locomotion benchmarks. With this starting point, achieving a specific gait characteristic requires iterations of heuristic reward-function adjustments, based on observed RL performance, until arriving at a desired behavior. This approach can be tedious when it works and is unreliable as a general framework. Other reference-free approaches structure the reward around a specific type of locomotion behavior [7] without being easily extended to other behaviors.

The first contribution of our work is to present a principled framework for designing reward functions that can naturally capture all of the periodic bipedal locomotion gaits. We are motivated by the fact that all common bipedal gaits can be defined by periodic *swing phases* (foot swinging in the air) and *stance phases* (foot planted on the ground) for each foot [8]. A fundamental distinction between swing and stance

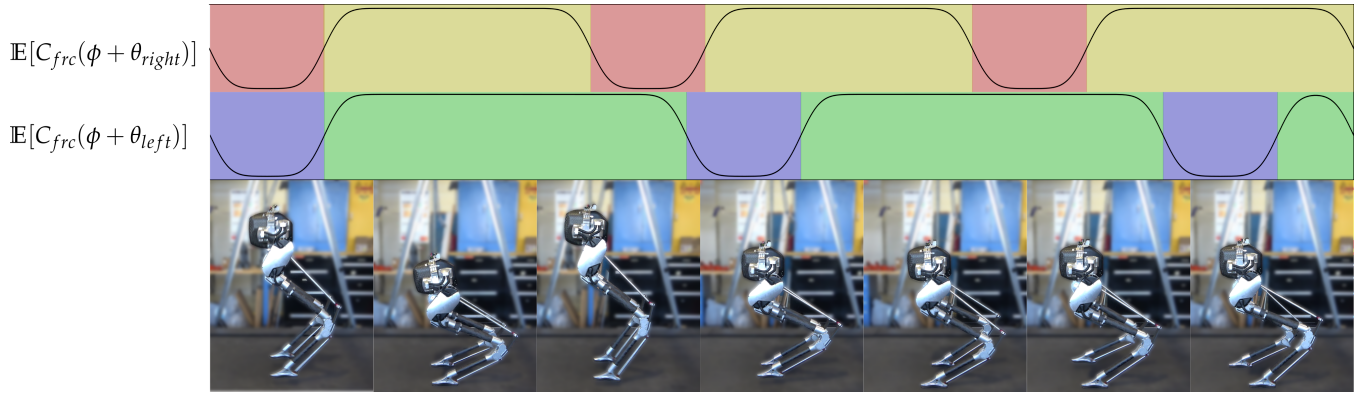


Fig. 2. A series of images showing a neural network policy controlling Cassie and continuously transitioning from hopping to galloping to walking. We present a simple reward design paradigm which makes use of probabilistic intervals to apply cost functions at specific times, allowing policies to learn all common bipedal gaits exhibited by animals in nature.

phases is the complementary presence and absence of foot forces and foot velocities for a given foot. We can create principled reward functions based on this observation by using the magnitudes of foot forces and velocities such that during a swing phase, forces are penalized while velocities are allowed, prompting the policy to learn to lift the foot. Thus, our framework describes gaits as a sequence of periodic phases, each of which rewards or penalizes a particular measurement of the physical system. Our hypothesis is that this framework will allow for a more natural specification of reward functions that sufficiently constrain RL to learn the desired gait characteristics, while allowing for flexible adaptation to specific environmental disturbances.

Our second contribution is to demonstrate this framework for sim-to-real RL of all common bipedal gaits, including walking, running, galloping, skipping, and hopping, without using a motion capture dataset or reference trajectories. We train policies for each of these behaviors in simulation and successfully demonstrate them on hardware. Further, by providing the framework's gait parameters as a command input to the policy, we are able to learn a multi-gait policy which can hop, gallop, run, and walk.

II. BACKGROUND

The practice of synthesizing locomotion behaviors has been studied in a variety of fields. In character animation, kinematics-based approaches (i.e., those using motion capture data) are commonly used to synthesize locomotion [9] [10] [11], with some newer approaches using deep learning [1] to train neural networks to solve problems like adapting realistically to varying ground geometries [12] [13] or blending several types of actions into one realistic body motion [2]. Physics-based character animation, wherein the pose of a character is controlled through the application of torques and motions are simulated using a physics engine, has also come into prominence in recent years [14], though is notably more difficult to use effectively for complicated motions [15] when compared to kinematics-based methods.

Approaches which use reinforcement learning to synthesize locomotion bear heavy similarities to the aforementioned

methods used in character animation. Much recent work uses trajectory-matching reward functions to train policies to imitate some reference trajectory (akin to a motion capture) while subjecting the policies to simulated physics, resulting in policies that behave realistically while imitating some reference trajectory [16] [5] [3] [17] [4]. Reinforcement learning has also been used for synthesizing locomotion without the use of reference trajectories, but these approaches often place little emphasis on subjective quality of behaviors, resulting in behaviors which maximize some objective while producing policies which are often inefficient, infeasible or unsafe to execute in the real world, and not usually visually pleasing [6] [18]. Methods which do prioritize physically realistic behavior without using a reference trajectory exist [19] [20] [7], but their reward functions are specific to single behaviors and are not trivial to extend to other behaviors.

III. LEARNING BIPEDAL GAITS WITH PERIODIC REWARD COMPOSITION

A. Reinforcement Learning Framework

We formulate our problem in the framework of reinforcement learning (RL) [21], for which we assume basic familiarity. The world is modeled as a discrete-time Markov Decision Process (MDP) with continuous state space $\mathcal{S}$, continuous action space $\mathcal{A}$, transition function $T(s, a, s')$, and reward function $R(s, t)$. Here $T(s, a, s')$ gives the probability density over the next state s' after taking action a in state s , and $R(s, t)$ gives the non-stationary reward for being in state s at time step t .

A control policy is a possibly stochastic mapping $\pi(a | s)$ from states to actions, which dictates behavior. Given a policy the expected T -horizon discounted return is given by $J(\pi) = \mathbb{E}[\sum_{t=0}^T \gamma^t R(S_t, t)]$, where $\gamma \in [0, 1]$ is a discount factor and S_t is a random variable representing the state at time t when following policy π under transition dynamics T . The goal of RL is to learn a policy π that maximizes $J(\pi)$ based on trial-and-error training experience in the world. In this work, we follow a sim-to-real RL paradigm where

training is done in simulation to identify a policy, which is then used in the real-world.

B. Periodic Reward Composition

Since our framework is targeted toward periodic behaviors, we index time via a *cycle time* ϕ variable, which repeatedly cycles over a normalized time period of $[0, 1]$ at discrete time steps rather than increasing monotonically. Accordingly, the non-stationary reward function $R(s, \phi)$ is periodic and defined in terms of ϕ rather than absolute time.

As motivated in the introduction, our framework specifies rewards in terms of compositions of rewards on periodic intervals. We define the reward as a biased sum of n *reward components* $R_i(s, \phi)$, where each component $R_i(s, \phi)$ captures a desired characteristic of the gait during a particular phase.

$$R(s, \phi) = \beta + \sum_i R_i(s, \phi)$$

Each reward component $R_i(s, \phi)$ is a product of a *phase coefficient* c_i , a *phase indicator* $I_i(\phi)$, and a real-valued *phase reward measurement* $q_i(s)$ (e.g. norm of a foot force).

$$R_i(s, \phi) = c_i \cdot I_i(\phi) \cdot q_i(s)$$

The phase coefficient c_i is a scalar whose sign determines the effect that the phase measurement $q_i(s)$ has on the total reward during cycle times when the reward component is active. The phase indicator function $I_i(\phi)$ is a binary-valued random variable denoting whether the target phase is active or not at cycle time ϕ . In this work, the distribution of each I_i is described by random variables A_i and B_i representing the start and end times of the period respectively. Since A_i and B_i represent intervals on a cycle, we use Von Mises distributions (approximations of the wrapped Normal distribution) described by the parameter tuple (a_i, b_i, κ) , which gives the means a_i and b_i and shared variance parameter κ . The distribution of the binary phase indicator $I_i(\phi)$ is then simply:

$$P(I_i(\phi) = x) = \begin{cases} P(A_i < \phi < B_i) & \text{if } x = 1 \\ 1 - P(A_i < \phi < B_i) & \text{if } x = 0 \end{cases} \quad (1)$$

where $P(A_i < \phi < B_i) = P(A_i < \phi)(1 - P(B_i < \phi))$

$$A_i \sim \Phi(2\pi a_i, \kappa) \text{ and } B_i \sim \Phi(2\pi b_i, \kappa)$$

Φ is the Von Mises distribution

Note that this formulation allows for uncertainty about the exact start and end of each phase to be captured via the variance parameter κ of the Von Mises distributions. This has a smoothing effect on the reward function at phase boundaries, which we have found to usefully encourage more stable and consistent learning. While we could directly use this probabilistic reward function $R(s, \phi)$ for RL training by sampling from the reward distribution at each step, we instead apply RL to the deterministic expectation of $R(s, \phi)$.

$$\mathbb{E}[R(s, \phi)] = \sum_i^n c_i \cdot \mathbb{E}[I_i(\phi)] \cdot q_i(s) + \beta$$

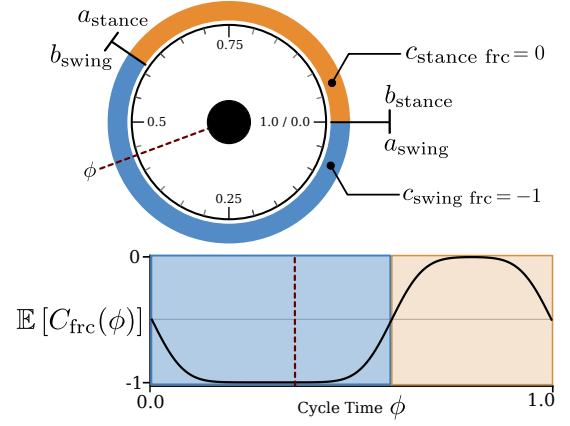


Fig. 3. The circular intervals of two phases, swing and stance, are shown on a polar plot. The expected sum of the phase indicators and phase coefficients for foot force $C_{\text{frc}}(\phi)$ is shown below.

Due to the linearity of expectations, for any policy π the expected cumulative return $J(\pi)$ is the same regardless of whether we use stochastic rewards or their expectations.

C. Describing Bipedal Gaits

For simplicity, we begin by describing repeatedly lifting and placing a single foot, or equivalently, cycling between swing and stance phases with our framework. As our phase reward measurements, we select the norm of foot force $q_{\text{frc}}(s)$ and the norm of foot velocity $q_{\text{spd}}(s)$. During the swing phase we want to penalize foot forces and ignore foot velocities, so we choose $c_{\text{swing frc}} = -1$ and $c_{\text{swing spd}} = 0$. Similarly, we choose $c_{\text{stance spd}} = -1$ and $c_{\text{stance frc}} = 0$ to penalize foot velocities and ignore foot forces during the stance phase. We constrain the swing and stance phases to follow immediately after one another and together last the entire cycle time by defining a ratio $r \in (0, 1)$ and setting the intervals for both phases such that the swing phase lasts length r , while the stance phase lasts length $1 - r$ and starts directly afterwards. Finally, by choosing a common scale κ , we can define the indicator functions $I_{\text{frc}}(\phi)$ and $I_{\text{spd}}(\phi)$ by using Equation 1.

For convenience, we define $C_{\text{frc}}(\phi)$ and $C_{\text{spd}}(\phi)$ as the expected sum of the product of all the phase indicators and phase coefficients for the swing and stance phases:

$$\begin{aligned} \mathbb{E}[C_{\text{frc}}(\phi)] &= c_{\text{swing frc}} \cdot \mathbb{E}[I_{\text{swing frc}}(\phi)] \\ &\quad + c_{\text{stance frc}} \cdot \mathbb{E}[I_{\text{stance frc}}(\phi)] \\ \mathbb{E}[C_{\text{spd}}(\phi)] &= c_{\text{swing spd}} \cdot \mathbb{E}[I_{\text{swing spd}}(\phi)] \\ &\quad + c_{\text{stance spd}} \cdot \mathbb{E}[I_{\text{stance spd}}(\phi)] \end{aligned}$$

Refer to Fig.3 for a visual explanation of the phase start and end times, ϕ , and the expected value of C_{frc} . For more complicated behaviors, we find that visualizing $C_{\text{frc}}(\phi)$ can be useful for understanding how the reward function changes over the cycle to guide the learning of a particular behavior.

Putting the force and speed components together, the expected overall reward for repeatedly lifting and placing a single foot is

$$\mathbb{E}[R_{\text{unipedal}}(s, \phi)] = \mathbb{E}[C_{\text{frc}}(\phi)] \cdot q_{\text{frc}}(s) + \mathbb{E}[C_{\text{spd}}(\phi)] \cdot q_{\text{spd}}(s)$$

Bipedal gaits are behaviors where the left and right feet both follow the same sequence of phases described above, but offset relative to each other in phase time. For instance, in a walking behavior the timings of the swing and stance phases are shifted apart by half of the period length (one leg in swing, the other in stance), while a hopping behavior is one where both feet synchronously enter the swing and stance phases. To expand the simple behavior of lifting and placing a single foot into the full spectrum of bipedal gaits, we need only introduce two *cycle offset* parameters $\theta_{\text{left}}, \theta_{\text{right}}$ which define the exact timing shift between the identical sequence of behavioral phases for the left and right feet, and differentiate between the norms of left and right foot forces, $q_{\text{left frc}}(s)$, $q_{\text{right frc}}(s)$ and norms of left and right foot velocities $q_{\text{left spd}}(s)$, $q_{\text{right spd}}(s)$. The expected overall reward for a bipedal behavior is

$$\begin{aligned} \mathbb{E}[R_{\text{bipedal}}(s, \phi)] = & \mathbb{E}[C_{\text{frc}}(\phi + \theta_{\text{left}})] \cdot q_{\text{left frc}}(s) \\ & + \mathbb{E}[C_{\text{frc}}(\phi + \theta_{\text{right}})] \cdot q_{\text{right frc}}(s) \\ & + \mathbb{E}[C_{\text{spd}}(\phi + \theta_{\text{left}})] \cdot q_{\text{left spd}}(s) \\ & + \mathbb{E}[C_{\text{spd}}(\phi + \theta_{\text{right}})] \cdot q_{\text{right spd}}(s) \end{aligned} \quad (2)$$

Introducing $\theta_{\text{left}}, \theta_{\text{right}}$ for two phase behaviors allows us to define reward functions for walking, galloping, and hopping. Additionally, by using four phases rather than two in each component, we can derive a skipping reward function, as shown in Fig. 4.

IV. METHOD

Network Architecture and Action Space: For all policies, we use a Long Short-Term Memory neural network [22] with two recurrent hidden layers of size 128 each, and a simple linear output projection of size 30, corresponding to 10 desired joint positions, and 10 sets of PD gains, as seen in Fig. 1. The desired joint positions are added to a set of constant offsets corresponding to a neutral standing position, such that an output vector of zeroes results in a standing pose. Similarly, the P gains and D gains are also summed with a fixed 'neutral' set of gains. The policy is evaluated at 40Hz, and the robot's PD controllers are run at 2000Hz. A similar system is used in [3] and [4], though without PD gain deltas.

State Space: To prevent the reinforcement learning environment from becoming a non-stationary Markov decision process, some information about the periodic reward function must be present in the state provided to the policy. Specifically, the policy must receive some sort of encoding of the current cycle time ϕ , an encoding of the cycle offset parameters, $\theta_{\text{left}}, \theta_{\text{right}}$, and information about the start and end times of the phases of the phase reward components.

In order to encode the current cycle time ϕ and the cycle offset parameters $\theta_{\text{left}}, \theta_{\text{right}}$, we condition the policies on two clock inputs:

$$p = \{\sin(2\pi(\phi + \theta_{\text{left}})), \sin(2\pi(\phi + \theta_{\text{right}}))\}$$

To encode information about the start and end times of the phases into the state, we derive a vector of *ratios* from the sequence of phase timings; each ratio represents the proportion out of the total period that a phase occupies.

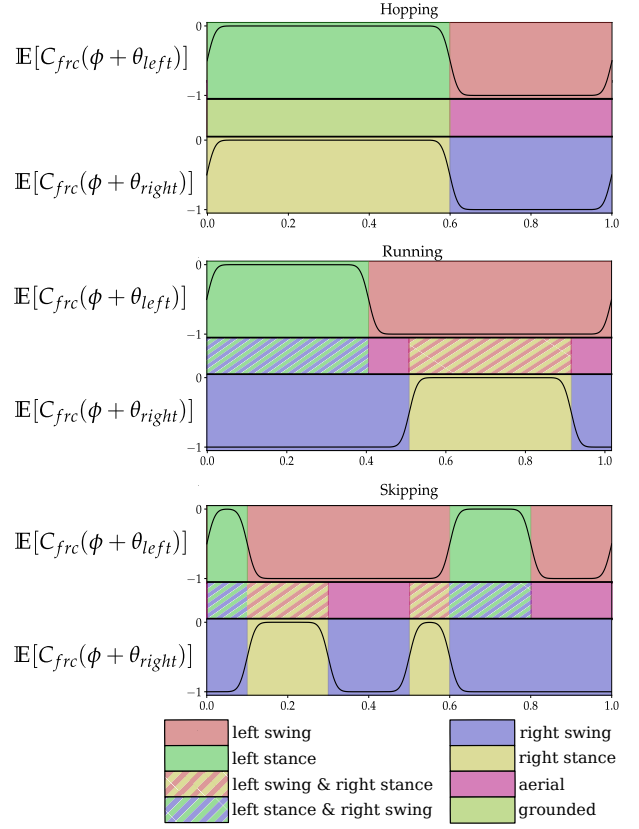


Fig. 4. Plot of the expected value of the force phase coefficient for each foot for some example bipedal gaits. In hopping there is a $|\theta_{\text{left}} - \theta_{\text{right}}| \approx 0$ shift between the left and right feet. For walking and running, $|\theta_{\text{left}} - \theta_{\text{right}}| \approx 0.5$, with transitional shifts resulting in galloping. Four phases are necessary for describing skipping, as well as a $|\theta_{\text{left}} - \theta_{\text{right}}| \approx 0.5$ shift between the left and right feet. The shaded region between the phase coefficient plots show the hybrid phases from combining the left and right foot behaviors together.

For instance, a phase j with start time $a_{i,j}=0.3$ and end time $b_{i,j}=0.7$ should occupy 40% of the total period time (plus or minus some uncertainty); thus, we can derive a ratio $r_j = 0.4$. Each phase's ratio is calculated and provided to the policy.

Thus, the policy's input consists of:

$$X_t = \begin{cases} \hat{q}, \hat{\dot{q}} & \text{robot state} \\ \dot{x}_{\text{desired}}, \dot{y}_{\text{desired}} & \text{desired velocity} \\ r, p & \text{phase ratios and clock inputs} \end{cases}$$

Where $\hat{q}, \hat{\dot{q}}$ are estimates of the pelvis orientation, rotational velocity, joint positions and joint velocities. $\dot{x}_{\text{desired}}$ and $\dot{y}_{\text{desired}}$ are speed commands randomized during training, and manually controlled by the user during evaluation.

Reward Formulation: We use the set of reward components from R_{bipedal} defined in Equation 2 to learn single-gait policies. In addition, we also use a variety of auxiliary cost components to further constrain the path of viable exploration towards stable locomotion and facilitate successful sim-to-real transfer as well as command the policy to match a desired orientation, forward speed, and sidespeed. These rewards do not need to vary as a function of time, and can be seen as a special case of the reward component framework, wherein the random variable c_i has only one phase, and thus

one possible value (in this case, -1 , to show that we wish to penalize these quantities q_i for the entire period).

$$\begin{aligned} R_{\text{cmd}}(s) &= (-1) \cdot q_{\dot{x}}(s) \\ &+ (-1) \cdot q_{\dot{y}}(s) \\ &+ (-1) \cdot q_{\text{orientation}}(s) \\ R_{\text{smooth}}(s) &= (-1) \cdot q_{\text{action diff}}(s) \\ &+ (-1) \cdot q_{\text{torque}}(s) \\ &+ (-1) \cdot q_{\text{pelvis acc}}(s) \end{aligned}$$

$$\mathbb{E}[R(s, \phi)] = \mathbb{E}[R_{\text{bipedal}}(s, \phi)] + R_{\text{smooth}}(s) + R_{\text{cmd}}(s) + \beta \quad (3)$$

Where $q_{\dot{x}}$ and $q_{\dot{y}}$ are measures of error between the commanded velocity and the actual pelvis velocity, and $q_{\text{orientation}}$ is the negative exponent of quaternion difference between the pelvis orientation and an orientation which faces straight, which can be used to change the heading of the robot. $q_{\text{pelvis acc}}$ is a cost for aggressive pelvis motion, which helps reduce noise in the state estimator, while $q_{\text{action diff}}$ is a cost for aggressive actions and q_{torque} is an overall joint torque cost to encourage efficient motions. For general policies which can learn and transition between a continuum of bipedal gaits, we specify additional reward terms which are discussed in Section V.

Dynamics Randomization: In order to facilitate successful sim-to-real transfer, we use dynamics randomization [23] [24] to expose the policies to a wide variety of possible real-world dynamics. Specifically, we randomize the execution rate of the policy, the mass and damping of the joints, the friction of the ground, the slope of the ground, and joint position encoder noise. Details on the ranges and distributions used to randomize these parameters can be found in Table I. These parameters are randomized at the beginning of every rollout during training and remain constant throughout each rollout.

Parameter	Unit	Range
Joint damping	Nms/rad	$[0.3, 4.0] \times \text{default values}$
Joint mass	kg	$[0.5, 1.5] \times \text{default values}$
Ground Friction	–	$[0.35, 1.1]$
Ground Slope	rad	$[-0.03, 0.03]$
Joint Encoder Offset	rad	$[-0.05, 0.05]$

TABLE I. The ranges for randomization of several dynamics parameters during training. We use a uniform distribution over the given ranges for all listed parameters.

Proximal Policy Optimization: To train our policies, we use a common model-free reinforcement learning algorithm known as Proximal Policy Optimization (PPO) [25]. More specifically, we use the recurrent adaptation described in [4], which samples batches of trajectories from a replay buffer, rather than batches of individual timesteps. In our case, we use a recurrent critic with exactly the same dimensions as the actor with the exception of the final linear output vector, which in the case of the critic is a scalar. We use a fixed exploration action noise, with standard deviation e^{-1} . In

addition, we incorporate a mirror loss term [26] into our policy gradient algorithm with the aim of achieving more symmetric locomotion behavior.

V. EXPERIMENTAL RESULTS

Training Details. Policies were trained using PPO with a batch size of 32 trajectories of up to 300 timesteps each, a learning rate of 0.0001 for both the actor and critic, a replay buffer of 50,000 samples, and 4 epochs per iteration. Training was terminated after 150,000,000 samples, which took between 24 and 36 hours per policy. We use the *cassie-mujoco-sim* [27] simulator, based on MuJoCo [28], to simulate Cassie during training.

Single-Gait Policy Results. We found that it was straightforward to use the probabilistic framework to train policies to learn standalone gaits, such as hopping, walking, running, or even skipping. These behaviors can be learned simply by holding the values of ratios r and cycle offset parameters $\theta_{\text{left}}, \theta_{\text{right}}$ to be constant throughout training (we refer the reader to Fig. 4 for examples of gait specifications). Videos of single-gait policies which learn skipping hopping, and walking can be found in our submission video.

Multi-Gait Policy Results By keeping the cycle offsets fixed and varying the phase ratios over some range during training, we are able to train policies that can learn to hop with more or less time in the air, and policies that can transition from walking to running. However, learning to transition between gaits by varying both the cycle offsets and phase ratios during training appears to be a challenging learning problem: policies which are trained in this fashion can end up asymmetrically walking instead of hopping, or learn other undesirable behaviors that resemble a fusion of all the different commanded gaits. To avoid these issues, we introduce additional reward terms called *transition penalties* to further distinguish each of the desired behaviors from each other during training. Transition penalties are behavior-specific costs which are only active when specific behaviors are being commanded. For hopping, which is commanded when cycle offsets are very close to each other, we activate a cost for constraining the feet positions to be close together. Similarly, by adding another transition penalty for standing, we can train a generic controller to transition between all steady state two-beat gaits and standing in place. Full details of the reward function for generic controllers are described in detail in the Appendix.

Outdoor Experiments We demonstrate the robustness of policies learned with our approach in a variety of outdoor experiments, shown in our submission video ¹. Given that the robot is effectively blind to the external world, the behaviors we observe in response to disturbances are surprisingly robust. We show the multi-gait policy hopping on and off a sidewalk, walking with one foot elevated on a curb, and running over small bumps in its path. The policy is also shown smoothly transitioning between all of the 2-beat gaits while moving forward on a crowned road, and while turning

¹youtu.be/4DnxV9lko-U

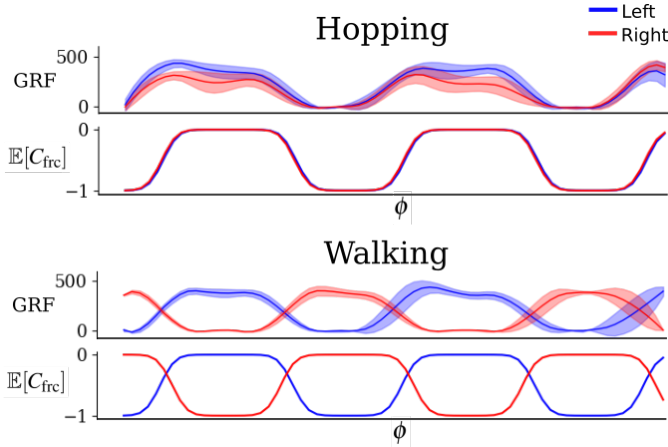


Fig. 5. Comparison between the mean measured simulation ground reaction forces (GRFs) in newtons and the corresponding expected values of C_{frc} over multiple policies. When the expectation is close to -1, the policies refrain from applying foot forces. When the expectation is close to 0, the policies apply foot forces.

in a small circle on a turf field. We also show a multi-gait policy descending down 5 steps of stairs while in a running gait. In the staircase and sidewalk tests, we observe the foot slipping off of small ledges and being compliant to the slope of the ground. This indicates a priority on force profile, as opposed to position only.

VI. CONCLUSION

In this work, we introduced a probabilistic reward framework which allows for learning of all of the common bipedal gait behaviors observed in animals. This framework requires no reference trajectories, enabling policies to explore a rich space of possible interaction with the world during training without artificial constraints to some arbitrary trajectory through space. We showed that not only are we able to train policies to learn all common bipedal gaits individually, including walking, running, hopping, galloping, and skipping, but also that we can learn all 2-beat behaviors on single policies and transition between them continuously, even while in motion. Some questions which remain unanswered include how easily this framework might be applied to the space of possible quadrupedal gaits, or other bipedal morphologies. A modified version of this framework could also be applied to aperiodic motions to learn one-off behaviors.

APPENDIX

Full Reward Functions for Generic 2-Beat Policies

When learning the continuum of 2-beat gaits, we find that policies often learn to stutter rather than hop with their feet together, which is not observed when training hopping standalone. To combat this, we introduce a *hop symmetry* term, a cost which becomes active when the cycle offsets of both legs match closely and punishes large distances between the feet in the sagittal and transverse planes, err_{sym} ,

$$q_{hop\ sym}(s) = 1 - \exp(-err_{sym} \exp(-5 |\sin(2\pi(\theta_{left} - \theta_{right}))|))$$

To learn a standing behavior in addition to the rest of the 2-beat gaits, the reward function must be modified to reflect our wish for the policy to remain very still when commanded.

First, we define the standing region of gait parameter space as one where the swing-to-stance phase ratio is close to 0.

We define $\omega = (1 + \exp(-50(r_{swing} - 0.15)))^{-1}$, which is a coefficient close to one during normal locomotion (when the swing and stance ratios are in the range [0.35, 0.7], and close to zero during standing (where the swing phase ratio is close to zero). Now we define an additional standing cost, which applies an additional action difference cost and a foot symmetry cost (similar to the hopping symmetry) when the command inputs are in the standing region.

$$q_{standing\ cost}(s) = 1 - \exp(-((err_{sym}) + 20q_{action\ diff}(s)))$$

Now we define the full reward for a generic policy which includes standing:

$$\begin{aligned} \mathbb{E}[R_{multi}(s, \phi)] = & 0.400 \cdot \mathbb{E}[R_{bipedal}(s, \phi)] \\ & + 0.300 \cdot \mathbb{E}[R_{cmd}(s)] \\ & + 0.100 \cdot \mathbb{E}[R_{smooth}(s)] \\ & + 0.100 \cdot (\omega - 1) \cdot q_{standing\ cost}(s) \\ & + 0.100 \cdot (-1) \cdot q_{hop\ sym}(s) \\ & + 1 \end{aligned}$$

Quantity q_i	Detail
$q_{left/right\ frc}$	$1 - \exp(-\omega \ raw_foot_frc\ _2^2 / 100)$
$q_{left/right\ spd}$	$1 - \exp(-2 \cdot \omega \ raw_foot_spd\ _2^2)$
$q_{\dot{x}}$	$1 - \exp(-2 \cdot \omega \dot{x}_{desired} - \dot{x}_{actual})$
$q_{\dot{y}}$	$1 - \exp(-2 \cdot \omega \dot{y}_{desired} - \dot{y}_{actual})$
$q_{orientation}$	$1 - \exp(-3 \cdot (1 - ((quat_{actual})^T (quat_{des})))^2)$
$q_{action\ diff}$	$1 - \exp(-5 \cdot \ a_t - a_{t-1}\)$
q_{torque}	$1 - \exp(-0.05 \cdot \ \tau\)$
$q_{pelvis\ acc}$	$1 - \exp(-0.10 \cdot (\ pelvis_{rot}\ + \ pelvis_{acc}\))$

TABLE II.

Where $\dot{x}_{actual}$ and $\dot{y}_{actual}$ are the current forward and lateral speeds of the pelvis. $quat_{actual}$ and $quat_{des}$ are the actual pelvis orientation and the desired pelvis orientation, in quaternion format. a_t is the current timestep's action, and a_{t-1} is the previous timestep's action. τ is the net torque applied to all joints across the robot. $pelvis_{rot}$ is the rotational velocity of the pelvis, and $pelvis_{acc}$ is the translational acceleration of the pelvis. Note that certain terms are multiplied by ω , signifying that these terms should not be respected by the policy when $\omega \approx 0$ (i.e., the policy is somewhere inside the standing region of the gait parameter space), to prevent the policy from (for example) attempting to match a desired speed while standing. The choice of $1 - \exp(-|x|)$ as a kernel function was influenced by a desire to have a reward bounded by 0 and 1.

ACKNOWLEDGEMENTS

Thanks to Jeremy Dao, Helei Duan, and Kevin Green for technical advice, and to Intel for access to the vLab academic compute cluster. This work is supported by the NSF Grant No. IIS-1849343 and DARPA Contract W911NF-16-1-0002.

REFERENCES

- [1] H. Zhang, S. Starke, T. Komura, and J. Saito, "Mode-adaptive neural networks for quadruped motion control," *ACM Transactions on Graphics*, vol. 37, no. 4, pp. 1–11, 2018, ISSN: 15577368. DOI: 10.1145/3197517.3201366.
- [2] S. Starke, Y. Zhao, T. Komura, and K. Zaman, "Local motion phases for learning multi-contact character movements," *ACM Transactions on Graphics*, vol. 39, no. 4, 2020, ISSN: 0730-0301. DOI: 10.1145/3386569.3392450.
- [3] Z. Xie, P. Clary, J. Dao, P. Morais, J. Hurst, and M. van de Panne, "Learning locomotion skills for cassie: Iterative design and sim-to-real," L. P. Kaelbling, D. Kragic, and K. Sugiura, Eds., ser. *Proceedings of Machine Learning Research*, vol. 100, PMLR, 2020, pp. 317–329. [Online]. Available: <http://proceedings.mlr.press/v100/xie20a.html>.
- [4] J. Siekmann, S. Valluri, J. Dao, L. Bermillo, H. Duan, A. Fern, and J. Hurst, "Learning Memory-Based Control for Human-Scale Bipedal Locomotion," 2020. arXiv: 2006.02402. [Online]. Available: <http://arxiv.org/abs/2006.02402>.
- [5] X. B. Peng, P. Abbeel, S. Levine, and M. van de Panne, "Deep-Mimic," *ACM Transactions on Graphics*, vol. 37, no. 4, pp. 1–14, 2018, ISSN: 0730-0301. DOI: 10.1145/3197517.3201311.
- [6] G. Brockman, V. Cheung, L. Pettersson, J. Schneider, J. Schulman, J. Tang, and W. Zaremba, "Openai gym," *arXiv preprint arXiv:1606.01540*, 2016.
- [7] J. Hwangbo, J. Lee, A. Dosovitskiy, D. Bellicoso, V. Tsounis, V. Koltun, and M. Hutter, "Learning agile and dynamic motor skills for legged robots," *Science Robotics*, vol. 4, no. 26, 2019.
- [8] Z. Gan, Y. Yesilevskiy, P. Zaytsev, and C. D. Remy, "All common bipedal gaits emerge from a single passive model," *Journal of The Royal Society Interface*, vol. 15, no. 146, p. 20180455, 2018.
- [9] S. Levine, J. M. Wang, A. Haraux, Z. Popovic, and V. Koltun, "Continuous character control with low-dimensional embeddings," *ACM Transactions on Graphics (TOG)*, vol. 31, no. 4, pp. 1–10, 2012.
- [10] A. Safonova, J. K. Hodgins, and N. S. Pollard, "Synthesizing physically realistic human motion in low-dimensional, behavior-specific spaces," *ACM Transactions on Graphics (TOG)*, vol. 23, no. 3, pp. 514–521, 2004.
- [11] P.-B. Wieber and C. Chevallereau, "Online adaptation of reference trajectories for the control of walking systems," *Robotics and Autonomous Systems*, vol. 54, no. 7, pp. 559–566, 2006.
- [12] D. Holden, T. Komura, and J. Saito, "Phase-functioned neural networks for character control," *ACM Transactions on Graphics*, vol. 36, no. 4, pp. 1–13, 2017, ISSN: 15577368. DOI: 10.1145/3072959.3073663.
- [13] S. Starke, H. Zhang, T. Komura, and J. Saito, "Neural state machine for character-scene interactions," 2019.
- [14] S. Agrawal, S. Shen, and M. van de Panne, "Diverse motion variations for physics-based character animation," *Symposium on Computer Animation*, 2013.
- [15] T. Geijtenbeek, N. Pronost, A. Egges, and M. H. Overmars, "Interactive character animation using simulated physics."
- [16] K. Bergamin, S. Clavet, D. Holden, and J. R. Forbes, "Drecon: Data-driven responsive control of physics-based characters," *ACM Transactions on Graphics (TOG)*, vol. 38, no. 6, pp. 1–11, 2019.
- [17] X. B. Peng, G. Berseth, and M. Van de Panne, "Terrain-adaptive locomotion skills using deep reinforcement learning," *ACM Transactions on Graphics (TOG)*, vol. 35, no. 4, pp. 1–12, 2016.
- [18] Y. Tassa, Y. Doron, A. Muldal, T. Erez, Y. Li, D. de Las Casas, D. Budden, A. Abdolmaleki, J. Merel, A. Lefrancq, T. P. Lillicrap, and M. A. Riedmiller, "Deepmind control suite," *CoRR*, vol. abs/1801.00690, 2018. arXiv: 1801.00690. [Online]. Available: <http://arxiv.org/abs/1801.00690>.
- [19] T. Haarnoja, S. Ha, A. Zhou, J. Tan, G. Tucker, and S. Levine, "Learning to walk via deep reinforcement learning," *arXiv preprint arXiv:1812.11103*, 2018.
- [20] J. Tan, T. Zhang, E. Coumans, A. Iscen, Y. Bai, D. Hafner, S. Bohez, and V. Vanhoucke, "Sim-to-real: Learning agile locomotion for quadruped robots," in *Proceedings of Robotics: Science and Systems*, Pittsburgh, Pennsylvania, Jun. 2018. DOI: 10.15607/RSS.2018.XIV.010.
- [21] R. S. Sutton and A. G. Barto, *Reinforcement learning: An introduction*. MIT press, 2018.
- [22] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural computation*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [23] X. B. Peng, M. Andrychowicz, W. Zaremba, and P. Abbeel, "Sim-to-real transfer of robotic control with dynamics randomization," in *2018 IEEE International Conference on Robotics and Automation (ICRA)*.
- [24] J. Tan, T. Zhang, E. Coumans, A. Iscen, Y. Bai, D. Hafner, S. Bohez, and V. Vanhoucke, "Sim-to-real: Learning agile locomotion for quadruped robots."
- [25] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, *Proximal Policy Optimization Algorithms*, 2017. arXiv: 1707.06347 [cs.LG]. [Online]. Available: <https://arxiv.org/abs/1707.06347>.
- [26] F. Adbolhosseini, H. Y. Ling, Z. Xie, X. B. Peng, and M. van de Panne, "On learning symmetric locomotion," in *Proc. ACM SIGGRAPH Motion, Interaction, and Games (MIG 2019)*, 2019.
- [27] Agility Robotics, *Cassie-mujoco-sim*, 2018. [Online]. Available: <https://github.com/osudrl/cassie-mujoco-sim>.
- [28] E. Todorov, T. Erez, and Y. Tassa, "Mujoco: A physics engine for model-based control," in *2012 IEEE/RSJ International Conference on Intelligent Robots and Systems*, IEEE, 2012, pp. 5026–5033.